AI-Powered Employee Onboarding Portal

Full Stack Project Document for Fresher Engineers

8-Day Execution Plan | Industry-Oriented | Beginner-Friendly

Prepared by: Manav Bharatiya | Wipro

1. Project Objective

1.1 What Is This Project?

The AI-Powered Employee Onboarding Portal is a web application that guides newly hired employees through every step of their joining process — from document submission and IT asset allocation to training assignments and buddy pairing. HR admins manage the entire onboarding workflow, and an AI chatbot answers new-hire questions about company policies, tools, and processes instantly.

Think of it as a digital welcome kit — instead of new employees receiving a pile of forms and endless emails on Day 1, they log in to a single portal that shows them exactly what to do, step by step, and lets them ask questions 24/7 through an AI assistant.

1.2 Business Use Case

In IT companies like Wipro, hundreds of freshers join every month. Managing their onboarding manually — distributing forms, tracking document submission, assigning laptops, scheduling trainings — is chaotic and error-prone. This portal automates and centralises all of it:

- New employees see a clear onboarding checklist on Day 1 — no confusion
- HR tracks onboarding completion status for every new hire in real time
- IT team assigns and tracks assets (laptop, ID card, access credentials)
- Training modules are assigned and completion is tracked automatically
- AI chatbot answers 'How do I submit my documents?' or 'What is the WFH policy?' instantly
- Buddy pairing assigns an experienced employee to guide each new hire

1.3 Expected Outcome

- A working full-stack employee onboarding portal with role-based access
- AI chatbot integrated with onboarding policy FAQs using RAG
- Containerized with Docker, orchestrated with Kubernetes
- Automated CI/CD with Jenkins, full source code on GitHub

2. Technology Stack

Layer	Technology	Purpose
Frontend	React JS	Onboarding checklist UI, document upload, training tracker
Backend	FastAPI (Python)	Onboarding workflow APIs, task management, asset assignment
AI Service	Flask (Python)	RAG chatbot for onboarding policy Q&A and guidance
Database	Oracle Database	Employees, tasks, documents, assets, trainings, buddies
Containerization	Docker	Package all three services into portable containers
Orchestration	Kubernetes (K8s)	Deploy, manage, and scale containers in production
CI/CD	Jenkins	Automate build → test → Docker build → K8s deploy
Version Control	GitHub	Source code management, branching, pull requests
AI / LLM API	OpenAI / Groq API	Generate helpful onboarding guidance and policy answers
IDE	VS Code + GitHub Copilot	Code editor with AI-powered autocompletion

3. System Architecture

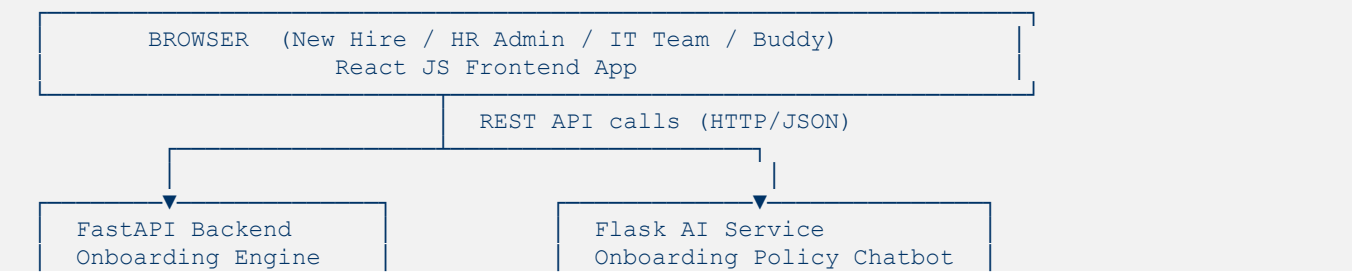
3.1 Simple Explanation

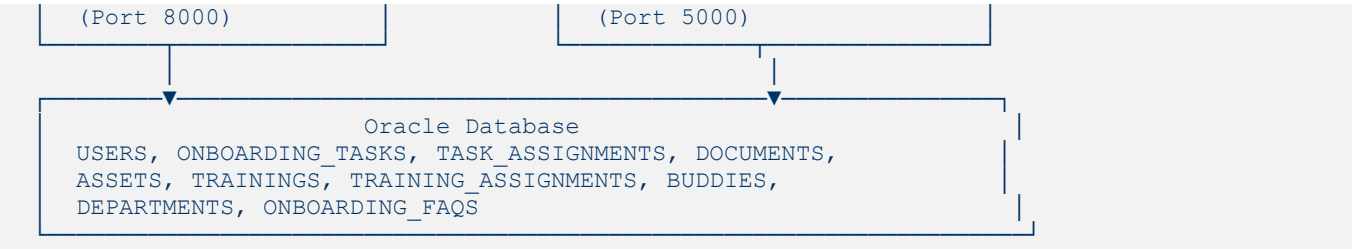
The system is split into three independent services:

- React Frontend — What the new hire, HR admin, IT team, and buddy see in the browser
- FastAPI Backend — Core onboarding logic: task assignments, document tracking, asset allocation
- Flask AI Service — Answers onboarding questions using RAG over company policy FAQs

All three run in Docker containers managed by Kubernetes. Jenkins automates the full deployment pipeline on every GitHub push to the main branch.

3.2 ASCII Architecture Diagram





3.3 DevOps Layer

- Docker: Each service (React, FastAPI, Flask) runs in its own container
- Kubernetes: Manages all pods — auto-restart, scaling, load balancing
- Jenkins: Code push to GitHub → tests → Docker build → deploy to K8s automatically

4. Project Modules

Module 1: Login / Signup

- Four roles: new_hire, hr_admin, it_admin, buddy
- HR admin creates new hire accounts on joining date
- JWT-based authentication, bcrypt password hashing
- Role-based dashboard redirect after login

Frontend Pages

- Login Page
- First-Time Password Setup Page (for new hires)

Backend APIs

- POST /auth/login
- POST /auth/register
- POST /auth/set-password
- GET /auth/me

Database Tables

- USERS (user_id, name, email, password_hash, role, department_id, joining_date, is_active, created_at)

Module 2: Onboarding Checklist & Task Management

- Each new hire gets a personalised onboarding checklist on Day 1
- Tasks are assigned based on department and role (e.g., IT tasks differ from HR tasks)
- Tasks have categories: Document Submission, IT Setup, Training, Policy Reading, Admin
- New hire marks tasks as complete; HR sees overall completion percentage
- Overdue tasks are highlighted automatically

Frontend Pages

- New Hire Dashboard with Checklist

- Task Detail Page
- HR Onboarding Tracker (all employees)

Backend APIs

- GET /tasks/my
- PUT /tasks/{id}/complete
- GET /admin/onboarding-status
- POST /admin/assign-tasks/{user_id}

Database Tables

- ONBOARDING_TASKS (task_id, title, description, category, default_due_days, is_mandatory, department_id)
- TASK_ASSIGNMENTS (assignment_id, task_id, user_id, status, due_date, completed_at, notes)

Module 3: Document Management

- New hire uploads required joining documents (Aadhaar, PAN, degree certificate, bank details, passport photo)
- HR admin reviews each document and marks it as Verified or Rejected with a reason
- New hire can re-upload rejected documents
- HR sees document completion status across all new hires

Frontend Pages

- My Documents Page with Upload Slots
- Document Upload Form
- HR Document Review Dashboard

Backend APIs

- GET /documents/my
- POST /documents/upload
- PUT /documents/{id}/verify
- PUT /documents/{id}/reject

Database Tables

- DOCUMENTS (doc_id, user_id, doc_type, file_path, status, reviewed_by, rejection_reason, uploaded_at, reviewed_at)

Module 4: IT Asset Allocation

- IT admin assigns assets to new hires (laptop, ID card, access card, mouse, keyboard)
- Each asset has a serial number, condition, and assignment date
- New hire confirms receipt of each assigned asset
- IT admin can recall assets when an employee exits

Frontend Pages

- My Assets Page
- IT Admin Asset Assignment Form

- Asset Inventory Dashboard

Backend APIs

- GET /assets/my
- POST /admin/assets/assign
- PUT /assets/{id}/confirm
- GET /admin/assets/inventory

Database Tables

- ASSETS (asset_id, name, serial_number, category, condition, status, created_at)
- ASSET_ASSIGNMENTS (aa_id, asset_id, user_id, assigned_by, assigned_date, confirmed_at, returned_at)

Module 5: Training Assignment & Tracker

- HR assigns mandatory training modules to new hires (Code of Conduct, Security Awareness, Technical Orientation, etc.)
- Each training has a title, description, duration, and a URL/resource link
- New hire marks training as completed and HR verifies
- Training completion is part of the onboarding checklist

Frontend Pages

- My Trainings Page with Progress Bar
- Training Detail Page with Resource Link
- HR Training Completion Report

Backend APIs

- GET /trainings/my
- PUT /trainings/{id}/complete
- POST /admin/trainings/assign
- GET /admin/trainings/report

Database Tables

- TRAININGS (training_id, title, description, duration_hours, resource_url, is_mandatory, created_at)
- TRAINING_ASSIGNMENTS (ta_id, training_id, user_id, status, assigned_at, completed_at)

Module 6: Buddy Pairing System

- HR pairs each new hire with an experienced employee (buddy) from the same team
- Buddy can see their assigned new hires and their onboarding progress
- New hire sees their buddy's contact details and can reach out directly
- Buddy logs check-in notes after weekly sessions with the new hire

Frontend Pages

- My Buddy Page (new hire view)
- My New Hires Page (buddy view)
- Check-in Notes Form

Backend APIs

- GET /buddy/my
- POST /admin/buddy/assign
- POST /buddy/checkin
- GET /buddy/checkins/{user_id}

Database Tables

- BUDDIES (buddy_id, new_hire_id, buddy_user_id, assigned_by, assigned_date, is_active)
- BUDDY_CHECKINS (checkin_id, buddy_id, notes, checkin_date, created_at)

Module 7: AI Onboarding Policy Chatbot

- New hire asks: 'What documents do I need to submit?' or 'What is the WFH policy?'
- Flask searches ONBOARDING_FAQS table for relevant policy answers
- New hire's current task status is also fetched for personalised responses
- LLM generates a friendly, helpful answer using the FAQ context

Frontend Pages

- Floating Chat Widget on all pages
- Full-screen Chat Page

Backend APIs

- POST /chatbot/ask (Flask AI Service)

Database Tables

- ONBOARDING_FAQS (faq_id, question, answer, category, keywords)

Module 8: HR Admin Panel & Reports

- HR sees total new hires, completion rates, pending documents, overdue tasks at a glance
- Bulk assign onboarding tasks to a department batch of new hires
- Export onboarding status report as CSV
- Manage departments, task templates, and training catalogue

Frontend Pages

- HR Admin Dashboard with KPI Cards
- Bulk Task Assignment Panel
- Reports & Export Page
- Department Management

Backend APIs

- GET /admin/dashboard-stats
- POST /admin/bulk-assign
- GET /admin/reports/export
- GET/POST /admin/departments

5. Database Design

5.1 Oracle Table Structures

Table: USERS

Column	Data Type	Constraints	Description
user_id	NUMBER	PRIMARY KEY	Unique user ID (sequence)
name	VARCHAR2(100)	NOT NULL	Full name
email	VARCHAR2(150)	UNIQUE, NOT NULL	Work email
password_hash	VARCHAR2(255)	NOT NULL	bcrypt hashed password
role	VARCHAR2(20)	DEFAULT 'new_hire'	new_hire / hr_admin / it_admin / buddy
department_id	NUMBER	FOREIGN KEY → DEPARTMENTS	Employee's department
joining_date	DATE		Official joining date
is_active	NUMBER(1)	DEFAULT 1	1=active, 0=deactivated
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation date

Table: DEPARTMENTS

Column	Data Type	Constraints	Description
department_id	NUMBER	PRIMARY KEY	Unique department ID
name	VARCHAR2(100)	NOT NULL	Engineering, HR, Finance, Operations
description	VARCHAR2(255)		Department description
head_user_id	NUMBER	FOREIGN KEY → USERS	Department head / manager

Table: ONBOARDING_TASKS

Column	Data Type	Constraints	Description
task_id	NUMBER	PRIMARY KEY	Unique task template ID
title	VARCHAR2(200)	NOT NULL	Task title
description	CLOB		Detailed task instruction
category	VARCHAR2(50)	NOT NULL	Document/IT Setup/Tra
default_due_days	NUMBER	DEFAULT 7	Days after joining to co
is_mandatory	NUMBER(1)	DEFAULT 1	1=mandatory, 0=option

Column	Data Type	Constraints	Description
department_id	NUMBER	FOREIGN KEY → DEPARTMENTS	NULL means applies to

Table: TASK_ASSIGNMENTS

Column	Data Type	Constraints	Descrip
assignment_id	NUMBER	PRIMARY KEY	Unique
task_id	NUMBER	FOREIGN KEY → ONBOARDING_TASKS	Task ter
user_id	NUMBER	FOREIGN KEY → USERS	New hir
status	VARCHAR2(20)	DEFAULT 'Pending'	Pending Comple
due_date	DATE	NOT NULL	Task du default_
completed_at	TIMESTAMP		When th complet
notes	VARCHAR2(300)		New hir

Table: DOCUMENTS

Column	Data Type	Constraints
doc_id	NUMBER	PRIMARY KEY
user_id	NUMBER	FOREIGN KEY → USERS
doc_type	VARCHAR2(100)	NOT NULL
file_path	VARCHAR2(500)	NOT NULL
status	VARCHAR2(20)	DEFAULT 'Pending'
reviewed_by	NUMBER	FOREIGN KEY → USERS
rejection_reason	VARCHAR2(300)	
uploaded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
reviewed_at	TIMESTAMP	

Table: ASSETS

Column	Data Type	Constraints	Descript
asset_id	NUMBER	PRIMARY KEY	Unique as
name	VARCHAR2(100)	NOT NULL	Laptop / I

Column	Data Type	Constraints	Descript
serial_number	VARCHAR2(100)	UNIQUE	Asset ser
category	VARCHAR2(50)	NOT NULL	Hardware Peripher
condition	VARCHAR2(20)	DEFAULT 'Good'	New / Go
status	VARCHAR2(20)	DEFAULT 'Available'	Available

Table: ASSET_ASSIGNMENTS

Column	Data Type	Constraints	Descri
aa_id	NUMBER	PRIMARY KEY	Unique
asset_id	NUMBER	FOREIGN KEY → ASSETS	Asset b
user_id	NUMBER	FOREIGN KEY → USERS	New hir
assigned_by	NUMBER	FOREIGN KEY → USERS	IT admi
assigned_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Assignm
confirmed_at	TIMESTAMP		When n receipt
returned_at	TIMESTAMP		When a

Table: TRAININGS

Column	Data Type	Constraints	Descr
training_id	NUMBER	PRIMARY KEY	Unique
title	VARCHAR2(200)	NOT NULL	Trainin
description	VARCHAR2(500)		What t
duration_hours	NUMBER(4,1)	NOT NULL	Estima
resource_url	VARCHAR2(500)		Link to LMS
is_mandatory	NUMBER(1)	DEFAULT 1	1=man

Table: TRAINING_ASSIGNMENTS

Column	Data Type	Constraints	Descri
ta_id	NUMBER	PRIMARY KEY	Unique ID
training_id	NUMBER	FOREIGN KEY → TRAININGS	Trainin
user_id	NUMBER	FOREIGN KEY → USERS	New hi

Column	Data Type	Constraints	Descri
status	VARCHAR2(20)	DEFAULT 'Pending'	Pendin Comple
assigned_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When a
completed_at	TIMESTAMP		When r

Table: BUDDIES

Column	Data Type	Constraints	Descri
buddy_id	NUMBER	PRIMARY KEY	Unique
new_hire_id	NUMBER	FOREIGN KEY → USERS	The new
buddy_user_id	NUMBER	FOREIGN KEY → USERS	The ass (experie
assigned_by	NUMBER	FOREIGN KEY → USERS	HR adm pairing
assigned_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date of
is_active	NUMBER(1)	DEFAULT 1	1=activ

Table: BUDDY_CHECKINS

Column	Data Type	Constraints	Description
checkin_id	NUMBER	PRIMARY KEY	Unique check-in ID
buddy_id	NUMBER	FOREIGN KEY → BUDDIES	Related buddy pairing
notes	CLOB	NOT NULL	Buddy's notes from the session
checkin_date	DATE	NOT NULL	Date of the buddy session
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time

Table: ONBOARDING_FAQS

Column	Data Type	Constraints	Description
faq_id	NUMBER	PRIMARY KEY	Unique FAQ ID
question	VARCHAR2(500)	NOT NULL	Onboarding-related question
answer	CLOB	NOT NULL	Detailed policy answer
category	VARCHAR2(100)		Documents / IT / Policy / Training / General

Column	Data Type	Constraints	Description
keywords	VARCHAR2(300)		Search keywords for RAG matching

5.2 ER Diagram (Text)

```
USERS ──────────────────────────────────────────────────────────────┐
  │                                                                 │
  ├── TASK_ASSIGNMENTS  (user_id FK, task_id FK)                    │
  ├── DOCUMENTS         (user_id FK, reviewed_by FK)                │
  ├── ASSET_ASSIGNMENTS (user_id FK, assigned_by FK)                │
  ├── TRAINING_ASSIGNMENTS (user_id FK, training_id FK)             │
  ├── BUDDIES           (new_hire_id FK, buddy_user_id FK)          │
  └── BUDDY_CHECKINS    (buddy_id FK)                               │
                                                                    │
DEPARTMENTS ──── USERS (department_id FK)                           │
DEPARTMENTS ──── ONBOARDING_TASKS (department_id FK)                │
ONBOARDING_TASKS ── TASK_ASSIGNMENTS (task_id FK)                   │
ASSETS ── ASSET_ASSIGNMENTS (asset_id FK)                           │
TRAININGS ── TRAINING_ASSIGNMENTS (training_id FK)  ────────────────┘
ONBOARDING_FAQS  (standalone - searched by AI chatbot)
```

5.3 Sample SQL — Seed Data

```
-- Departments
INSERT INTO DEPARTMENTS VALUES (1,'Engineering', 'Tech & Dev teams',     NULL);
INSERT INTO DEPARTMENTS VALUES (2,'HR',          'Human Resources',      NULL);
INSERT INTO DEPARTMENTS VALUES (3,'Finance',     'Finance & Accounts',   NULL);
-- Onboarding Task Templates
INSERT INTO ONBOARDING_TASKS
(task_id,title,category,default_due_days,is_mandatory,department_id)
VALUES (1,'Submit Aadhaar & PAN Card','Document',2,1,NULL);
VALUES (2,'Collect Laptop from IT',  'IT Setup', 1,1,NULL);
VALUES (3,'Complete Security Awareness Training','Training',5,1,NULL);
VALUES (4,'Read and Sign Code of Conduct','Policy',3,1,NULL);
VALUES (5,'Set Up Corporate Email',  'IT Setup', 1,1,NULL);
-- Mandatory Trainings
INSERT INTO TRAININGS VALUES (1,'Code of Conduct','Company ethics and behaviour
policy',1,'http://lms/coc',1,CURRENT_TIMESTAMP);
INSERT INTO TRAININGS VALUES (2,'Security Awareness','Data and cyber security
training',2,'http://lms/sec',1,CURRENT_TIMESTAMP);
```

6. API Documentation

6.1 Authentication APIs

Endpoint	Method	Description	Request Body	Response
/auth/login	POST	Login and get JWT	{email, password}	{access_token, role, user_id}
/auth/register	POST	HR creates new hire	{name, email, role, department_id, joining_date}	{message, user_id, temp_password}

Endpoint	Method	Description	Request Body	Response
/auth/set-password	POST	New hire sets own password	{user_id, new_password}	{message}
/auth/me	GET	Get logged-in user info	Header: Bearer token	{user_id, name, role, department}

6.2 Task & Checklist APIs

Endpoint	Method	Description	Auth Required
/tasks/my	GET	Get my onboarding checklist	New Hire
/tasks/{id}/complete	PUT	Mark a task as completed	New Hire
/admin/onboarding-status	GET	See all new hires and their % complete	HR Admin
/admin/assign-tasks/{user_id}	POST	Auto-assign tasks to a new hire	HR Admin
/admin/bulk-assign	POST	Bulk assign tasks to all dept new hires	HR Admin

6.3 Document APIs

Endpoint	Method	Description	Auth Required
/documents/my	GET	Get my document submission status	New Hire
/documents/upload	POST	Upload a document (multipart form)	New Hire
/documents/{id}/verify	PUT	HR marks document as Verified	HR Admin
/documents/{id}/reject	PUT	HR rejects with reason	HR Admin
/admin/documents	GET	All documents across all new hires	HR Admin

6.4 Asset & Training APIs

Endpoint	Method	Description	Auth Required
/assets/my	GET	Get my assigned assets	New Hire
/assets/{id}/confirm	PUT	Confirm receipt of asset	New Hire
/admin/assets/assign	POST	Assign asset to new hire	IT Admin

Endpoint	Method	Description	Auth Required
/admin/assets/inventory	GET	View all assets and their status	IT Admin
/trainings/my	GET	Get my assigned trainings	New Hire
/trainings/{id}/complete	PUT	Mark training as completed	New Hire
/admin/trainings/assign	POST	Assign training to user(s)	HR Admin
/admin/trainings/report	GET	Completion report for all trainees	HR Admin

6.5 Buddy & Chatbot APIs

Endpoint	Method	Description	Auth Required
/buddy/my	GET	New hire sees their buddy's details	New Hire
/admin/buddy/assign	POST	Pair a new hire with a buddy	HR Admin
/buddy/checkin	POST	Buddy logs a check-in note	Buddy
/buddy/checkins/{id}	GET	View all check-ins for a new hire	Buddy / HR Admin
/chatbot/ask	POST	Ask the onboarding AI assistant a question	All roles (Flask)

7. Frontend Structure (React JS)

7.1 Folder Structure

```
src/
├── components/
│   ├── Navbar.jsx              ← Top navigation bar
│   ├── Sidebar.jsx             ← Role-based side navigation
│   ├── TaskCard.jsx            ← Single checklist task card
│   ├── DocumentSlot.jsx        ← Upload slot for each document type
│   ├── AssetCard.jsx           ← Asset display with confirm button
│   ├── TrainingCard.jsx        ← Training module card with link
│   ├── ProgressBar.jsx         ← Onboarding completion progress bar
│   └── ChatWidget.jsx          ← Floating AI chatbot button
├── pages/
│   ├── Login.jsx               ← Login form
│   ├── SetPassword.jsx         ← First-time password setup
│   ├── NewHireDashboard.jsx    ← New hire home: checklist overview
│   ├── MyChecklist.jsx         ← Full onboarding checklist
│   ├── MyDocuments.jsx         ← Document upload and status
│   ├── MyAssets.jsx            ← Assigned assets + confirm receipt
│   ├── MyTrainings.jsx         ← Training modules + completion
│   ├── MyBuddy.jsx             ← Buddy details and contact info
│   ├── HRDashboard.jsx         ← HR KPI overview dashboard
```

```
│   ├── OnboardingTracker.jsx    ← All new hires completion status
│   ├── DocumentReview.jsx       ← HR document verification page
│   ├── BuddyManagement.jsx      ← HR buddy assignment panel
│   ├── ITAssetPanel.jsx         ← IT admin asset assignment
│   ├── BuddyDashboard.jsx       ← Buddy's view of assigned new hires
│   └── ChatPage.jsx             ← Full AI chatbot page
├── services/
│   ├── api.js                   ← Axios instance with JWT interceptor
│   ├── authService.js           ← Login/register/set-password calls
│   ├── taskService.js           ← Checklist and task API calls
│   ├── documentService.js       ← Document upload and verify calls
│   ├── assetService.js          ← Asset assignment and confirm calls
│   ├── trainingService.js       ← Training assignment and complete
│   ├── buddyService.js          ← Buddy pairing and check-in calls
│   └── chatService.js           ← Chatbot API calls
├── context/
│   └── AuthContext.jsx          ← Global user, role, token state
├── App.jsx                      ← All route definitions
└── index.jsx                    ← React entry point
```

7.2 Routing Table

Route	Component	Access
/	Login.jsx	Public
/set-password	SetPassword.jsx	New Hire (first login)
/dashboard	NewHireDashboard.jsx	New Hire
/checklist	MyChecklist.jsx	New Hire
/documents	MyDocuments.jsx	New Hire
/assets	MyAssets.jsx	New Hire
/trainings	MyTrainings.jsx	New Hire
/buddy	MyBuddy.jsx	New Hire
/hr	HRDashboard.jsx	HR Admin only
/hr/tracker	OnboardingTracker.jsx	HR Admin only
/hr/documents	DocumentReview.jsx	HR Admin only
/hr/buddies	BuddyManagement.jsx	HR Admin only
/it/assets	ITAssetPanel.jsx	IT Admin only
/buddy-dashboard	BuddyDashboard.jsx	Buddy only
/chat	ChatPage.jsx	All roles

8. Backend Structure (FastAPI)

8.1 Folder Structure

```
backend/
├── main.py                      ← FastAPI app, CORS middleware, routers
├── config.py                    ← DB_HOST, SECRET_KEY, FILE_UPLOAD_PATH
├── db.py                        ← Oracle DB connection (cx_Oracle pool)
├── models/
│   ├── user.py                  ← User Pydantic schemas
│   ├── task.py                  ← Task and assignment schemas
│   ├── document.py              ← Document upload schemas
│   ├── asset.py                 ← Asset schemas
│   ├── training.py              ← Training schemas
│   └── buddy.py                 ← Buddy pairing schemas
├── routes/
│   ├── auth.py                  ← /auth/* routes
│   ├── tasks.py                 ← /tasks/* routes
│   ├── documents.py             ← /documents/* routes
│   ├── assets.py                ← /assets/* routes
│   ├── trainings.py             ← /trainings/* routes
│   ├── buddies.py               ← /buddy/* routes
│   └── admin.py                 ← /admin/* routes
├── services/
│   ├── auth_service.py          ← JWT create/decode, bcrypt verify
│   ├── task_service.py          ← Auto-assign tasks on new hire creation
│   └── onboarding_service.py    ← Completion % calculation logic
├── uploads/                     ← Folder for uploaded document files
└── requirements.txt
```

8.2 Auto-Task Assignment on New Hire Creation

1. HR calls POST /auth/register to create a new hire with department_id and joining_date
2. After creating the USERS record, task_service.auto_assign() is called
3. It queries ONBOARDING_TASKS where department_id = new hire's dept OR department_id IS NULL
4. For each matching task template, calculate due_date = joining_date + default_due_days
5. INSERT a TASK_ASSIGNMENT row for each task with status='Pending'
6. Same logic applies for TRAINING_ASSIGNMENTS — mandatory trainings are auto-assigned
7. New hire logs in on Day 1 and immediately sees their complete onboarding checklist

8.3 Onboarding Completion % Calculation

8. Query: SELECT COUNT(*) FROM TASK_ASSIGNMENTS WHERE user_id=? → total_tasks
9. Query: SELECT COUNT(*) FROM TASK_ASSIGNMENTS WHERE user_id=? AND status='Completed' → done
10. Completion % = (done / total_tasks) × 100
11. This is shown as a progress bar on the new hire dashboard and HR tracker
12. HR sees a table of all new hires sorted by completion % ascending (most behind first)

9. AI Onboarding Chatbot — RAG Explained

9.1 What is RAG? (Simple)

RAG = Retrieval-Augmented Generation. Instead of only using the LLM's general knowledge, our chatbot first RETRIEVES relevant answers from the company's own onboarding FAQs stored in Oracle. It then passes that content to the LLM as context, so the response is specific to YOUR company's policies — not generic internet answers.

9.2 How the Onboarding Chatbot Works

13. New hire types: 'What documents do I need to submit in the first week?'
14. Flask AI service receives the question
15. Flask searches ONBOARDING_FAQS using keyword match on question and keywords columns
16. Optionally, Flask also fetches the new hire's pending tasks from TASK_ASSIGNMENTS
17. Both FAQ context and pending task list are combined into an LLM prompt
18. LLM generates a friendly, personalised response
19. Flask returns the answer to the React chat widget

9.3 RAG Flow Diagram

```
New Hire Question: 'What documents do I need to submit in the first week?'
        |
        ▼
[Flask AI Service]
   |                          |
   ├──Search──▶ [ONBOARDING_FAQS Table]   ← document policy FAQs
   |
   └──Fetch──▶  [TASK_ASSIGNMENTS Table]  ← new hire's pending tasks
        |
        ▼
Prompt = FAQ Policy Answer + Pending Tasks + New Hire Question
        |
        ▼
[LLM API]  →  'Please submit the following in Week 1: Aadhaar, PAN...'
        |
        ▼
Chat Widget → New hire sees a personalised, actionable response
```

9.4 Sample Flask Chatbot Code

```
from flask import Flask, request, jsonify
import cx_Oracle, openai, os

app = Flask(__name__)

def search_faqs(question):
    conn = cx_Oracle.connect(os.environ['DB_DSN'])
    cur  = conn.cursor()
    kw   = f'%{question[:25].lower()}%'
    cur.execute('''
        SELECT answer FROM ONBOARDING_FAQS
        WHERE LOWER(keywords) LIKE :kw OR LOWER(question) LIKE :kw
        AND ROWNUM <= 3''', kw=kw)
    return ' '.join(r[0] for r in cur.fetchall())

def get_pending_tasks(user_id):
    conn = cx_Oracle.connect(os.environ['DB_DSN'])
    cur  = conn.cursor()
```

```
    cur.execute('''
        SELECT t.title FROM TASK_ASSIGNMENTS ta
        JOIN ONBOARDING_TASKS t ON ta.task_id = t.task_id
        WHERE ta.user_id=:uid AND ta.status='Pending' ''', uid=user_id)
    return [r[0] for r in cur.fetchall()]

@app.route('/chatbot/ask', methods=['POST'])
def ask():
    question = request.json.get('question')
    user_id  = request.json.get('user_id')
    faq_ctx  = search_faqs(question)
    tasks    = get_pending_tasks(user_id)
    prompt   = (f'You are a friendly onboarding assistant.\n'
                f'Company policy: {faq_ctx}\n'
                f'Pending tasks: {tasks}\n'
                f'Question: {question}\nAnswer:')
    resp = openai.ChatCompletion.create(
        model='gpt-3.5-turbo',
        messages=[{'role':'user','content': prompt}])
    return jsonify({'answer': resp.choices[0].message.content})
```

9.5 Sample Onboarding FAQ Data

Question	Answer	Category
What documents do I need on Day 1?	Please bring Aadhaar card, PAN card, last degree certificate (original + copy), 2 passport photos, and bank account details for salary processing.	Documents
How do I get my laptop?	Visit the IT Help Desk on Floor 3 on Day 1 with your employee ID. Your laptop will be pre-configured. Collect it between 9 AM and 11 AM.	IT Setup
What is the WFH policy for new hires?	New hires are required to work from office for the first 90 days. After that, WFH is allowed up to 2 days per week with manager approval.	Policy
Who is my buddy and what do they do?	Your buddy is an experienced colleague assigned to help you settle in. They will check in with you weekly for the first 3 months. See the Buddy tab for their contact.	General

10. Docker Setup

10.1 Dockerfile — FastAPI Backend

```
FROM python:3.11-slim
WORKDIR /app
RUN mkdir -p /app/uploads
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

```
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

10.2 Dockerfile — React Frontend

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

10.3 Dockerfile — Flask AI Service

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

10.4 docker-compose.yml

```
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports: ['3000:80']
    depends_on: [backend]

  backend:
    build: ./backend
    ports: ['8000:8000']
    volumes:
      - ./uploads:/app/uploads
    environment:
      - DB_HOST=oracle-db
      - DB_PORT=1521
      - SECRET_KEY=onboard_secret_key

  ai-service:
    build: ./ai-service
    ports: ['5000:5000']
    environment:
      - OPENAI_API_KEY=your_api_key_here
      - DB_HOST=oracle-db
```

10.5 Docker Commands

Command	What it Does
docker-compose up --build	Build and start all 3 containers
docker-compose down	Stop and remove all running containers
docker ps	List all active containers
docker logs onboard-backend	View FastAPI backend logs
docker exec -it onboard-backend bash	Open a terminal inside the backend container
docker volume ls	List Docker volumes (used for uploaded files)

11. Kubernetes Setup

11.1 deployment.yaml — Backend

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: onboard-backend
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: onboard-backend
  template:
    metadata:
      labels:
        app: onboard-backend
    spec:
      containers:
      - name: backend
        image: onboard-backend:latest
        ports:
        - containerPort: 8000
        env:
        - name: SECRET_KEY
          value: 'onboard_secret_key'
        - name: DB_HOST
          value: 'oracle-service'
```

11.2 service.yaml — Backend

```
apiVersion: v1
kind: Service
metadata:
  name: onboard-backend-svc
spec:
  selector:
    app: onboard-backend
  ports:
  - protocol: TCP
    port: 8000
```

```
    targetPort: 8000
  type: ClusterIP
```

11.3 kubectl Commands

Command	What it Does
kubectl apply -f k8s/	Deploy all YAML files in the k8s folder at once
kubectl get pods	List all running pods and their status
kubectl get services	List all Kubernetes services
kubectl logs <pod-name>	View logs from a specific running pod
kubectl describe pod <pod-name>	Full diagnostic info about a pod
kubectl scale deployment onboard-backend --replicas=3	Scale backend to 3 running pods
kubectl rollout restart deployment/onboard-backend	Restart all backend pods (e.g. after config change)
kubectl port-forward svc/onboard-backend-svc 8000:8000	Forward K8s port to local machine for testing

12. Jenkins CI/CD Pipeline

12.1 Jenkinsfile

```
pipeline {
  agent any
  environment {
    APP = 'employee-onboarding-portal'
  }
  stages {
    stage('Checkout Code') {
      steps {
        git branch: 'main',
            url: 'https://github.com/your-org/onboarding-portal.git'
      }
    }
    stage('Install Dependencies') {
      steps {
        sh 'pip install -r backend/requirements.txt'
        sh 'pip install -r ai-service/requirements.txt'
        sh 'cd frontend && npm install'
      }
    }
    stage('Run Tests') {
      steps {
        sh 'cd backend && pytest tests/ -v --tb=short'
      }
    }
    stage('Build Docker Images') {
```

```
      steps {
        sh 'docker build -t onboard-backend:latest  ./backend'
        sh 'docker build -t onboard-frontend:latest ./frontend'
        sh 'docker build -t onboard-ai:latest       ./ai-service'
      }
    }
    stage('Deploy to Kubernetes') {
      steps {
        sh 'kubectl apply -f k8s/backend-deployment.yaml'
        sh 'kubectl apply -f k8s/frontend-deployment.yaml'
        sh 'kubectl apply -f k8s/ai-deployment.yaml'
        sh 'kubectl rollout status deployment/onboard-backend'
      }
    }
  }
  post {
    success { echo 'Onboarding Portal deployed successfully!' }
    failure { echo 'Build failed. Check Jenkins console output.' }
  }
}
```

13. Git Workflow

Branch	Purpose	Who Uses It
main	Final production-ready code only	Everyone (via approved PR)
develop	Integration branch — all features merge here	Everyone
feature/auth	Login, set-password, JWT, roles	Backend Developer
feature/tasks	Onboarding checklist and task assignment	Backend + Frontend Dev
feature/documents	Document upload, verify, reject flow	Backend + Frontend Dev
feature/assets	IT asset assignment and confirmation	Backend + Frontend Dev
feature/trainings	Training assignment and completion tracker	Backend + Frontend Dev
feature/buddy	Buddy pairing and check-in notes module	Backend + Frontend Dev
feature/chatbot	Flask AI chatbot, RAG, LLM integration	AI Engineer
feature/devops	Dockerfiles, K8s YAMLs, Jenkinsfile	DevOps Engineer

13.1 Commit Message Convention

Prefix	When to Use	Example
feat:	New feature added	feat: add auto task assignment on new hire register
fix:	Bug fixed	fix: correct onboarding completion percentage calc
docs:	Documentation updated	docs: add document upload API description
test:	Tests added	test: add unit tests for task_service.py
chore:	Config or tooling change	chore: add uploads volume to docker-compose

14. Team Roles & Responsibilities

Role	Responsibilities	Key Tech
Frontend Dev (1-2)	15 React pages, checklist UI, document upload, progress bar, chat widget, role routing	React, Axios, CSS, react-router-dom
Backend Dev (1-2)	FastAPI routes, auto-task assignment logic, completion %, document upload, Oracle queries	FastAPI, cx_Oracle, JWT, Python
Database Engineer (1)	10 Oracle tables, FK constraints, department + task + FAQ seed data, SQL queries, sequences	Oracle SQL, SQL*Plus, cx_Oracle
DevOps Engineer (1)	3 Dockerfiles, docker-compose with volume, K8s YAMLs for all services, 5-stage Jenkinsfile	Docker, Kubernetes, Jenkins
AI Engineer (1)	Flask chatbot, FAQ + pending task fetching, LLM prompt design, RAG flow, chatbot testing	Flask, OpenAI/Groq API, cx_Oracle
Docs & Presentation (1)	README, 12-slide PPT, 5-min demo video, viva Q&A prep, final code review and merge	PowerPoint, OBS Studio, GitHub

15. 8-Day Execution Plan

Day 1 — Setup, GitHub & Database

Item	Details
Tasks	Create GitHub repo with 10 branches, design Oracle schema for 10 tables, create all tables with sequences and FKs, insert seed data (departments, task templates, trainings, sample FAQs), install tools (Node, Python, VS Code, Oracle client)

Item	Details
Expected Output	GitHub repo live with 10 branches; Oracle DB running with all 10 tables created and seeded; all team members cloned and running locally with no errors
Responsible	DB Engineer (all table SQL + seed data), DevOps Engineer (GitHub repo + branch setup), All members (local environment install)

Day 2 — Authentication & User Module

Item	Details
Tasks	FastAPI register (HR creates new hire), login, set-password (first login), JWT with role+user_id, bcrypt, React Login page, SetPassword page, role-based redirect to correct dashboard after login
Expected Output	HR can register a new hire; new hire receives temp credentials; new hire can set their own password and log in; all 4 roles redirect to correct dashboards
Responsible	Backend Developer (auth APIs), Frontend Developer (login + set-password pages)

Day 3 — Task Checklist & Auto-Assignment

Item	Details
Tasks	Auto-task assignment logic on new hire creation (task_service.py), task listing API, mark-complete API, completion % calculation, React NewHireDashboard with ProgressBar, MyChecklist page with TaskCard components, HR OnboardingTracker
Expected Output	When HR registers a new hire, tasks are auto-assigned; new hire logs in and sees complete checklist; HR tracker shows all new hires sorted by completion %
Responsible	Backend Developer (task service + APIs), Frontend Developer (dashboard + checklist pages)

Day 4 — Document & Asset Module

Item	Details
Tasks	Document upload API (multipart file), verify/reject APIs, asset assignment and confirm APIs, asset inventory API, React MyDocuments with DocumentSlot components, MyAssets page, HR DocumentReview page, IT ITAssetPanel page
Expected Output	New hire can upload documents; HR can verify or reject with reason; IT admin can assign a laptop and new hire can confirm receipt

Item	Details
Responsible	Backend Developer (document + asset APIs), Frontend Developer (document + asset pages)

Day 5 — Training & Buddy Module

Item	Details
Tasks	Training assignment APIs, completion API, training report API, buddy pairing API, check-in API, React MyTrainings page with TrainingCard, MyBuddy page, HR BuddyManagement panel, Buddy BuddyDashboard with check-in form
Expected Output	New hire sees assigned trainings and can mark them complete; HR can pair buddies; buddy can log weekly check-in notes; HR sees buddy check-in history
Responsible	Backend Developer (training + buddy APIs), Frontend Developer (training + buddy pages)

Day 6 — AI Chatbot + Docker

Item	Details
Tasks	Flask AI service with FAQ search + pending task fetch + LLM API, React ChatWidget floating button and ChatPage, all 3 Dockerfiles, docker-compose.yml with uploads volume, test full stack running in Docker containers
Expected Output	Chatbot answers onboarding questions with personalised responses; all 3 services start successfully via docker-compose up --build; full application accessible from browser
Responsible	AI Engineer (Flask + RAG + LLM), Frontend Dev (chat widget + page), DevOps (Docker setup)

Day 7 — Kubernetes + Jenkins

Item	Details
Tasks	Write K8s deployment.yaml + service.yaml for all 3 services (6 YAML files), set up Jenkins server, write 5-stage Jenkinsfile, connect GitHub webhook to Jenkins, test full pipeline from push to K8s deployment
Expected Output	Jenkins completes all 5 stages without error; kubectl get pods shows all pods Running; services reachable via kubectl port-forward
Responsible	DevOps Engineer (leads all K8s + Jenkins work), all members verify their service deploys correctly

Day 8 — Testing, Bug Fixing & Final Presentation

Item	Details
Tasks	End-to-end testing of full new hire journey (register → checklist → documents → assets → trainings → buddy → chatbot), fix all reported bugs, merge all feature branches to main via PRs, record 5-min demo video, create 12-slide PPT, practice viva
Expected Output	Bug-free portal, polished 12-slide PPT, 5-min MP4 demo video, all code on main branch with clean commit history, full team confident for evaluation
Responsible	Documentation Owner (PPT + demo video recording), All team members (testing + viva Q&A prep)

16. Evaluation Criteria

Criteria	Marks	What Evaluators Look For
Code Quality	15	Clean, readable, well-commented code; good folder structure; no hardcoded values
Frontend (React)	10	15 working pages, role-based routing, progress bar, upload form, chat widget
Backend (FastAPI)	10	All APIs working, auto-task assignment logic, completion % calculation, auth
Database (Oracle)	10	10 tables with correct FKs, sequences, meaningful seed data, tested SQL queries
AI Chatbot (RAG)	10	Chatbot uses FAQ + pending tasks for personalised onboarding answers
Docker Setup	10	All 3 services run via docker-compose; uploads volume correctly mounted
Kubernetes	10	All pods running; deployment + service YAMLs correct; port-forward works
Jenkins Pipeline	10	All 5 stages complete without failure; rollout status verified
Git Usage	5	10 branches used, conventional commit messages, PRs with reviews before merge
Teamwork	5	All 6 members contributed; each person owns their module clearly
Presentation & PPT	5	Clear 12-slide deck, confident live demo, viva answers show understanding
TOTAL	100	

17. Final Deliverables

Deliverable	Description	Format
Source Code	All frontend, backend, AI service — all 8 modules complete	GitHub Repo
GitHub Repository	10 branches, commit history, all PRs merged to develop → main	GitHub URL
README.md	Project overview, setup steps, env variables, how to run	Markdown
Dockerfiles	3 Dockerfiles (React, FastAPI, Flask) tested and working	Dockerfile × 3
docker-compose.yml	Full-stack local config with uploads volume mount	YAML
K8s YAML Files	deployment.yaml + service.yaml per service in /k8s/ folder	YAML × 6
Jenkinsfile	5-stage CI/CD pipeline in project root	Jenkinsfile
PPT Presentation	12 professional slides with architecture and demo screenshots	PowerPoint
Demo Video	5-min screen recording: full new hire onboarding journey	MP4

18. Presentation Structure (PPT Guide)

Slide	Title	Content	Presenter
1	Team Introduction	Team name, 6 member names, individual roles assigned	All together
2	Problem Statement	Day-1 chaos for new hires: no direction, paper forms, missing assets	Docs Owner
3	Technology Stack	Table of 10 technologies with their specific purpose in this portal	Backend Dev
4	System Architecture	ASCII diagram, 3 services, Oracle DB with 10 tables, DevOps pipeline	Backend Dev
5	Database Design	10 Oracle tables, ER diagram, task template + FAQ seed data screenshot	DB Engineer
6	New Hire Journey Demo	Live: login → checklist → upload doc → confirm asset → complete training	Frontend Dev
7	HR & IT & Buddy Demo	Live: HR verifies doc → IT assigns laptop → buddy logs check-in note	Frontend Dev

Slide	Title	Content	Presenter
8	AI Chatbot Demo	Live: 'What docs do I need?' → show personalised answer with pending tasks	AI Engineer
9	Docker/K8s Demo	Show docker-compose up, kubectl get pods, explain 3-container architecture	DevOps Eng
10	Jenkins Pipeline	Open Jenkins → trigger build → show 5 stages passing → verify K8s deploy	DevOps Eng
11	Challenges & Learnings	File upload handling in Docker, cx_Oracle pool setup, auto-task logic bug	All (1 each)
12	Future Scope	Email/SMS alerts, mobile app, e-signature, LMS integration, exit management	Docs Owner

18.1 Presentation Tips for Freshers

- Open with energy: 'Good morning! We are Team [name], and this is our AI-Powered Employee Onboarding Portal.'
- For the new hire demo: walk through the complete journey as if YOU are the new hire joining today
- For the chatbot demo: ask 'What should I do on my first day?' — the personalised answer with pending tasks will impress evaluators
- Have docker-compose already running before the session — don't start it during the demo
- If a viva question stumps you: 'Great question — in our implementation we handled it by... but we know X is an alternative approach we'd explore next'

19. Viva Questions & Sample Answers

React JS

Question	Simple Answer
How did you handle file uploads in React?	We used an HTML input with type='file' inside a DocumentSlot component. On file selection, we used FormData to package the file and send it via Axios with Content-Type: multipart/form-data to the POST /documents/upload API.
How does the progress bar update in real time?	When a new hire completes a task (marks it done), we call the API and on success we re-fetch the task list using useEffect. The completion percentage is recalculated in the backend and returned. React re-renders the ProgressBar component with the new value.

Question	Simple Answer
How did you show different dashboards for 4 roles?	After login, the JWT payload contains the role. We store it in AuthContext. In App.jsx, we used a PrivateRoute wrapper that checks the role and either renders the correct page or redirects to /dashboard if the role doesn't match the route.

FastAPI / Backend

Question	Simple Answer
How does auto task assignment work?	When HR registers a new hire, we immediately call task_service.auto_assign(user_id, department_id, joining_date). This queries ONBOARDING_TASKS for that department (or NULL for all-department tasks) and inserts a TASK_ASSIGNMENT row for each, with due_date = joining_date + default_due_days.
How did you handle document file storage?	We used FastAPI's UploadFile type. The file is saved to a local /app/uploads folder with a unique filename (UUID + original extension). The file path is stored in the DOCUMENTS table. In Docker, the uploads folder is mounted as a volume so files persist even if the container restarts.
What is a connection pool and why did you use it?	A connection pool keeps a set of Oracle DB connections open and reuses them for API requests. Without it, every API call would open a new connection (slow). We used cx_Oracle.SessionPool to create a pool of 5 connections that are shared across all requests — much faster.

Oracle Database

Question	Simple Answer
Why does ONBOARDING_TASKS have a nullable department_id?	A NULL department_id means the task applies to ALL new hires regardless of department (e.g., 'Submit PAN card'). A non-NULL department_id means the task is specific to that department (e.g., 'Set up Dev environment' is only for Engineering).
How did you calculate task due dates in SQL?	Due date is calculated as: joining_date + default_due_days. In Oracle SQL: due_date = TRUNC(u.joining_date) + t.default_due_days, where u is USERS and t is ONBOARDING_TASKS. We do this at insert time when assigning tasks.
Why use a volume for file uploads in Docker?	Docker containers are stateless — when a container restarts, any files written inside it are lost. By mounting a host directory as a volume (./uploads:/app/uploads), uploaded documents are stored on the host machine and survive container restarts.

Docker, Kubernetes & Jenkins

Question	Simple Answer
What is the purpose of depends_on in docker-compose?	depends_on: [backend] tells Docker Compose to start the backend container before the frontend container. This prevents the React container from starting before the API it depends on is ready.
What is the difference between ClusterIP and NodePort in K8s?	ClusterIP exposes a service only inside the Kubernetes cluster — used for internal services like the backend and AI chatbot. NodePort exposes a service on a specific port on every node — used when external access is needed. For this project, we used ClusterIP and kubectl port-forward for testing.
How does Jenkins know when to run the pipeline?	We configured a GitHub Webhook on the repository. Every time code is pushed to the main branch, GitHub sends an HTTP POST request to our Jenkins server URL. Jenkins receives this webhook trigger and automatically starts the pipeline without any manual action needed.

GenAI / RAG

Question	Simple Answer
Why does your chatbot fetch the user's pending tasks?	Without the pending task list, the chatbot would give generic answers. By fetching the actual tasks that are still Pending for that specific user, the LLM can give a personalised response like 'You still need to: submit Aadhaar card, confirm laptop receipt' — much more useful.
What is the system role in an LLM prompt?	The system role sets the AI's behaviour and persona. We set it to 'You are a friendly onboarding assistant.' This tells the LLM to always respond in a helpful, supportive tone and stay focused on onboarding topics — it won't go off-topic or be too formal.
How would you improve RAG with semantic search?	Currently we use SQL LIKE keyword matching which misses synonyms (e.g., 'laptop' won't match 'computer'). With semantic search, we'd convert FAQ text to vector embeddings, store them in a vector DB like pgvector or Pinecone, and at query time convert the question to a vector and find the most semantically similar FAQs — much more accurate.

Best Of Luck!

Build with curiosity. Collaborate with respect. Deliver with pride.